A one-step test and the constructive generation of bit sequences with exponentially cumulated frequency distributions of their runs

The Evaluation of Finite Bit Sequences

Nicolas Windt

October 28, 2026

We introduce the *one-step test*, an evaluation procedure for finite bit sequences which cumulates the frequencies of the runs and sub-runs of a sequence. Applying it to the output of the random device `/dev/random` of a Linux installation yields an almost exactly decreasing exponential course of the cumulated frequencies, in accordance with the geometric distribution of run lengths of independent bits. Motivated by this observation, we study bit sequences whose cumulated frequency distribution falls off exactly exponentially and collect them into classes B_N. We show that the distribution of the runs without sub-runs (base distribution) of the elements of B_N satisfies a strict recursion, and we derive an identity for the number of offset sub-runs. Building on this, we give an iterative and a recursive algorithm for generating an element of B_N. Finally, we assign to every natural number a unique composition of pairwise distinct classes B_N, obtain from it a family of composite frequency distributions, and discuss the notion of randomness thereby induced.

Keywords: random sequence · run · runs test · cumulated frequency distribution · exponential distribution · random number generator · `/dev/random`

1. Introduction

The starting point of this work are Karl Popper's investigations of the concept of probability in "The Logic of Scientific Discovery" [8]: randomness is not a property that can be read off a single event, but an ascription tied to the repeatability and verifiability of observations. For the finite bit sequences treated here we draw the consequence not to presuppose randomness axiomatically, but to examine finite sequences by means of observable, purely combinatorial structures – without recourse to limit statements about infinite sequences.

In this work, first an evaluation procedure for finite bit sequences is designed, the one-step test (Section 4), and the result of applying it to the output of a random device is analysed (Section 5). The experiment will show that the frequency distribution of the runs and sub-runs of the output of a random generator approaches an exponentially shaped distribution. Motivated by this experimental result, the structure of the frequency distributions of such bit sequences is investigated formally, thereby explaining the experimental observation (Section 6). Finally, algorithms generating elements of the classes arising there are given (Section 7), and it is shown how these classes compose via the

binary representation of natural numbers (Section 8). Section 9 discusses the consequences for the concept of randomness; Section 10 summarises.

The contributions in detail:

- a linear evaluation procedure which cumulates the frequencies of runs *and* sub-runs of finite bit sequences (one-step test, Section 4);
- an experimental evaluation of the output of `/dev/random` (Section 5);
- the classes B_N of bit sequences with exactly exponentially decreasing cumulated frequency distribution, including the structure of their base distribution (Theorem 12) and an offsetting identity for sub-runs (Lemma 16);
- an iterative and a recursive generator for elements of B_N (Section 7);
- a one-to-one correspondence between natural numbers and compositions of pairwise distinct classes B_N (Section 8).

2. State of Research

The mathematical foundation of the concept of randomness is the Kolmogorov axiomatisation [4]; the strong and the weak law of large numbers following from it justify the expectation that in sufficiently long sequences of independent, uniformly distributed bits the run lengths are geometrically distributed: the probability of a run length k is 2^{-k} [1, and the exercises on runs therein]. The exponentially decreasing course of the (cumulated) frequencies observed here is therefore to be expected for ideal random sequences; what is new in this work is not the decrease itself but the cumulation over the sub-runs and the study of those finite sequences whose cumulated distribution falls off *exactly* exponentially.

Statistical procedures for testing random sequences have long been established. The classical runs test of Wald and Wolfowitz [10] tests the number of runs of a sequence against its expected distribution; to this day it is part of the standardised test batteries for cryptographic random generators, such as the NIST suite [9] and its predecessors [3, 6]. Surveys on the design and testing of random number generators are given by Knuth [3] and L'Ecuyer [5]. The algorithmic concept of randomness of Martin-Löf [7] defines randomness of infinite sequences via the impossibility of effective tests; closest to our approach is the idea there of capturing randomness by passing effective tests, with the difference that we consider finite sequences and a single, purely combinatorial quantity.

As experimental source we use the device `/dev/random` of a Linux installation; its way of operating is documented and has been analysed repeatedly [2]. The evaluation used here is independent of the particular source; we single out `/dev/random` merely as a notoriously good supplier of bit sequences whose origin is not to be explained causally.

The epistemological framing follows Popper [8]. We return to the question whether, and in which sense, the classes B_N can replace the probabilistic concept of randomness in Section 9.

3. Foundations: Runs and Sub-Runs

Definition 1 (Set of finite bit sequences)**.**

$$\mathcal{B} := \{(b_m)_{1 \le m \le M < \infty},\ b_m \in \{0, 1\}\} \tag{1}$$

We start from an arbitrary finite bit sequence

$$(b_m)_{1\leq m\leq M,\, M<\infty} \in \mathcal{B} \tag{2}$$

For example

$$1000101010011111000001111111000101010 1 \tag{3}$$

This bit sequence can also be viewed as a sequence of runs, where a run is a sequence of constant bits. In our example these are

$$1 \cdot 000 \cdot 1 \cdot 0 \cdot 1 \cdot 0 \cdot 1 \cdot 00 \cdot 11111 \cdot 00000 \cdot 1111111 \cdot 000 \cdot 1 \cdot 0 \cdot 1 \cdot 0 \cdot 1 \cdot 0 \cdot 1 \tag{4}$$

Definition 2 (Run)**.** A *run* of a bit sequence $(b_m)_{1\leq m\leq M} \in \mathcal{B}$ is a constant sub-sequence $(b_p, \ldots, b_q)$, maximal in both directions, with $1 \leq p \leq q \leq M$:

$$b_p = b_{p+1} = \cdots = b_q, \qquad (p = 1 \text{ or } b_{p-1} \neq b_p)\,, \qquad (q = M \text{ or } b_q \neq b_{q+1})\,. \tag{5}$$

The number $q - p + 1$ is called the *length* of the run.

Lemma 3 (Run decomposition)**.** *Every bit sequence $(b_m)_{1\leq m\leq M} \in \mathcal{B}$ decomposes uniquely into runs. With $m_1 := 1$ and the recursive rule*

$$m_{i+1} := \min\left(\{m > m_i :\ b_m \neq b_{m_i}\} \cup \{M+1\}\right) \tag{6}$$

one obtains unique decomposition points $1 = m_1 < m_2 < \cdots < m_N < m_{N+1} = M + 1$ with runs $(b_{m_i}, \ldots, b_{m_{i+1}-1})$ and run lengths $r_n := m_{n+1} - m_n$. In particular

$$\sum_{n=1}^{N} r_n = M, \tag{7}$$

and consecutive runs carry complementary bit values. The sequence $(r_n)_{1\leq n\leq N} \in \mathbb{N}$ is called the run-length sequence *of (b_m).*

Proof. Since (b_m) is constant on each block $[m_i, m_{i+1})$ and $b_{m_{i+1}} \neq b_{m_i}$ holds (provided $m_{i+1} \leq M$), (6) is well defined and yields a strictly increasing sequence which reaches $M + 1$ after finitely many steps. The maximality conditions of (5) ensure that each block is a run in the sense of the definition. Uniqueness follows because the first run must begin at $m = 1$ and every subsequent run is determined by the end of its predecessor. □

Remark 4 (Reversal and complement). Every bit sequence can be read from right to left as well as from left to right without changing its run-length sequence; without loss of generality we therefore always read sequences from left to right. The proof is trivial and will not be carried out here.

Furthermore, two bit sequences possess the same run-length sequence if and only if they are equal or binary complements of each other: together with the bit of the first run, the run-length sequence determines the bit sequence uniquely because of the alternating construction, and passing to the complement changes only that single bit.

Since only the lengths of the runs enter all frequency considerations of this work, we henceforth identify runs with their length and work with run-length sequences $(r_n) \in \mathbb{N}$.

Furthermore the trivial observation holds that a run of length N contains runs $s < N$ with the corresponding frequency distribution

$${}_N S = ({}_N s_n)_{0<n\leq N}, \quad \text{with } {}_N s_n = N - n + 1. \tag{8}$$

For illustration we take the run $N = 8$

$$1\,1\,1\,1\,1\,1\,1\,1 \quad \text{or} \quad 0\,0\,0\,0\,0\,0\,0\,0 \tag{9}$$

Then N contains:

$$\begin{aligned}
&{}_8s_1 = 8 - 1 + 1 = 8 \text{ runs } 1, && {}_8s_2 = 8 - 2 + 1 = 7 \text{ runs } 2,\\
&{}_8s_3 = 8 - 3 + 1 = 6 \text{ runs } 3, && {}_8s_4 = 8 - 4 + 1 = 5 \text{ runs } 4,\\
&{}_8s_5 = 8 - 5 + 1 = 4 \text{ runs } 5, && {}_8s_6 = 8 - 6 + 1 = 3 \text{ runs } 6,\\
&{}_8s_7 = 8 - 7 + 1 = 2 \text{ runs } 7, && {}_8s_8 = 8 - 8 + 1 = 1 \text{ run } 8.
\end{aligned}$$

Definition 5 (Sub-run)**.** The runs of smaller length contained in a run are henceforth called sub-runs.

4. The One-Step Test

We view the bit sequence under investigation as a sequence of runs

$$(r_n)_{0<n\leq N}, \quad r_n \in N, \quad N \text{ suitable} \tag{10}$$

To evaluate such a sequence we determine the cumulated distribution

$$L = (l_p)_{0<p\leq P<\infty} \tag{11}$$

of the frequencies l_p of the runs and of the sub-runs contained in them of the sequence $(r_n)_{0<n\leq N<\infty}$.

We motivate this approach by the following consideration: for the output $(b_m) \in \mathcal{B}$ of a random algorithm delivering independent, uniformly distributed bits, one has $P(b_{n-1} = b_n) = P(b_{n-1} \neq b_n) = \frac{1}{2}$ [1]. Instead of resorting to limit statements, we prefer to consider, for each bit b_m of (b_m), the frequency of the cases $(b_m) = (b_{m+i})$. This leads us to the approach of cumulated distributions: we view $(b_m) \in \mathcal{B}$ as a sequence (r_n) and determine the frequencies of its runs and of the sub-runs contained in them.

Trivially, the largest index P of the sequence (11) is given by the longest run of (r_n):

$$P = \max\{r_1, \ldots, r_N\} \tag{12}$$

The preceding considerations lead to the following procedure, to which we give the name one-step test.

```
Input: an arbitrary sequence of runs of length N, N finite
create an array L of length at most N
for each run of the sequence
    determine the length of the run (here l)
    for each position i = 1 to l
        increase the frequency at position i by l - i + 1
output L
```

Algorithm 1: One-step test for sequences of runs

The same procedure can be applied to the bit sequence (b_m) or $(\bar{b}_m)$ underlying the sequence of runs (r_n). A slightly different computational rule results; that it yields the same result as Algorithm 1 is shown in Lemma 6. The name "one-step test" derives from the fact that this version of the algorithm – the chronologically first – processes the input step by step; its inner loop runs for every single bit with the current counter value z.

```
Input: an arbitrary finite sequence of bits of length P, P finite
create an array L of length at most P
create a counter z with value 0
for each bit of the sequence (step by step)
    has the value of the bit changed compared to the previous step?
    if no, increase the value of counter z by 1
    if yes, reset counter z to 1
    for each position i = 1 to z    (for every single bit!)
        increase the frequency at position i by 1
output L
```

Algorithm 2: One-step test for bit sequences

Lemma 6 (Equivalence)**.** *Algorithms 1 and 2 yield the same cumulated distribution for every bit sequence.*

Proof. Consider a run of length l. Through this run, Algorithm 1 increases the frequency at position i ($1 \leq i \leq l$) by $l - i + 1$. Algorithm 2 has the counter value $z = j$ at the j-th bit of the run; this bit therefore increases positions 1 through j. Position i is thus increased by the bits $j = i, \ldots, l$, that is, exactly $l - i + 1$ times. Both algorithms enter the same contribution for each run, and the contributions of different runs add up independently. □

5. A Motivating Experiment

We resort to an experimental means in order to produce a bit sequence whose structure is not to be explained causally: a so-called random sequence. Such an experimental means is the device `/dev/random` of an arbitrary Linux installation [2], which enjoys the reputation of being a notoriously good supplier of such bit sequences, even though it works rather slowly. Moreover, its way of operating can be verified without gaps. We collected the output of `/dev/random` over a period of about fifteen months (November 13, 2016, to February 15, 2018) with a background script which runs permanently: as soon as 1024 bytes have been read from `/dev/random`, the file is written and the next block is read immediately after.

```
dd if=/dev/random of="$(date +%Y_%m_%d_%H_%M_%S)" bs=1024 count=1
```

The script is an endless loop that runs in the background and starts automatically at every system power-on. (Short read accesses of the entropy pool occasionally result in files smaller than 1024 bytes.) The result are 9 856 files totalling 9 793 146 bytes.[1] The result of this experimental procedure is a finite bit sequence whose length M is determined by the experiment:

$$(b_m)_{1 \leq m \leq M < \infty}, \quad b_m \in \{0, 1\}, \tag{13}$$

with $M = 78\,345\,168$ bits.
As set out in Section 3 on runs and sub-runs, we now view the sequence (b_m) of (13) as a sequence of the form (10) and evaluate it with the one-step test. The result of the evaluation of the output of the device `/dev/random` is recorded in Table 1. The distribution of the runs and sub-runs of the output of `/dev/random` falls off exponentially: $\log_2 l_n$ decreases by almost exactly 1 per step as n grows. We

[1] By the definition of the one-step test, l_1 counts every bit of the sequence exactly once; the consistency condition $l_1 = M = 78\,345\,168$ is checked automatically during the evaluation. The data set as well as the collection and evaluation scripts are contained in the repository (`dev/random` and `paper/experiment`).

present these results graphically in Figure 1; the theoretical series shown there and in Table 1 is the expectation $l_n = M \cdot 2^{1-n}$ for independent, uniformly distributed bits. A chi-square goodness-of-fit test of the distribution of the run lengths against the geometric distribution yields $\chi^2 = 19.8$ at 22 degrees of freedom (critical value at level 5%: 33.9); the estimate of the transition probability $\hat{p} = 0.49994 \pm 0.00006$ lies 1.05 standard deviations below $\frac{1}{2}$. The survey is thus compatible with independent, uniformly distributed bits. In the spirit of the intersubjective verification of this experiment, the reader is expressly encouraged to reproduce it by the means of his choice.

length	frequency	$\log_2$	expectation	length	frequency	$\log_2$	expectation
1	78345168	26.22	78345168	13	19282	14.23	19127
2	39177249	25.22	39172584	14	9588	13.23	9564
3	19592313	24.22	19586292	15	4753	12.21	4782
4	9798825	23.22	9793146	16	2364	11.21	2391
5	4900914	22.22	4896573	17	1174	10.20	1195
6	2451773	21.23	2448287	18	561	9.13	598
7	1227083	20.23	1224143	19	248	7.95	299
8	614018	19.23	612072	20	110	6.78	149
9	307704	18.23	306036	21	50	5.64	75
10	153948	17.23	153018	22	22	4.46	37
11	76935	16.23	76509	23	11	3.46	19
12	38572	15.24	38254	24	5	2.32	9

Table 1: Frequency l_n of the runs and sub-runs of the output of `/dev/random`, computed from the complete data set ($M = 78\,345\,168$ bits in 9 856 files; the longest run has length 25). The third column gives $\log_2 l_n$, the fourth the expectation $M \cdot 2^{1-n}$ (rounded) for independent, uniformly distributed bits.

This observation suggests that the output of `/dev/random` – or of any device serving the purpose of delivering bit sequences without causal generation – exhibits an almost exponentially decreasing course of the distribution of its runs and contained sub-runs. Contradictorily, this would explain a structure of chance. In the following sections we study such bit sequences without regard to the way they are obtained.

6. Bit Sequences with Exponentially Cumulated Frequency Distributions

6.1. The cumulated distribution and the classes B_N

Definition 7 (Cumulated distribution)**.** Let $({}_N b_m)$ be a bit sequence of $\mathcal{B}$ and $(r_n)_{0<n\leq N<\infty}$ its corresponding sequence of runs. We denote by the sequence

$$ {}_N L := ({}_N l_i)_{0<i\leq N<\infty} \tag{14}$$

the distribution of the frequencies of the runs and contained sub-runs of the bit sequence $({}_N b_m)$ and call it the cumulated distribution.

Definition 8 (Exponentially decreasing cumulated distribution)**.** By the exponentially decreasing cumulated distribution ${}_N L := (l_i)_{0<i\leq N<\infty}$ of a bit sequence $({}_N b_m) \in \mathcal{B}$ we understand

$$ \forall\, 0 < n \leq N < \infty : \quad {}_N l_n = 2^{N-n} \tag{15}$$

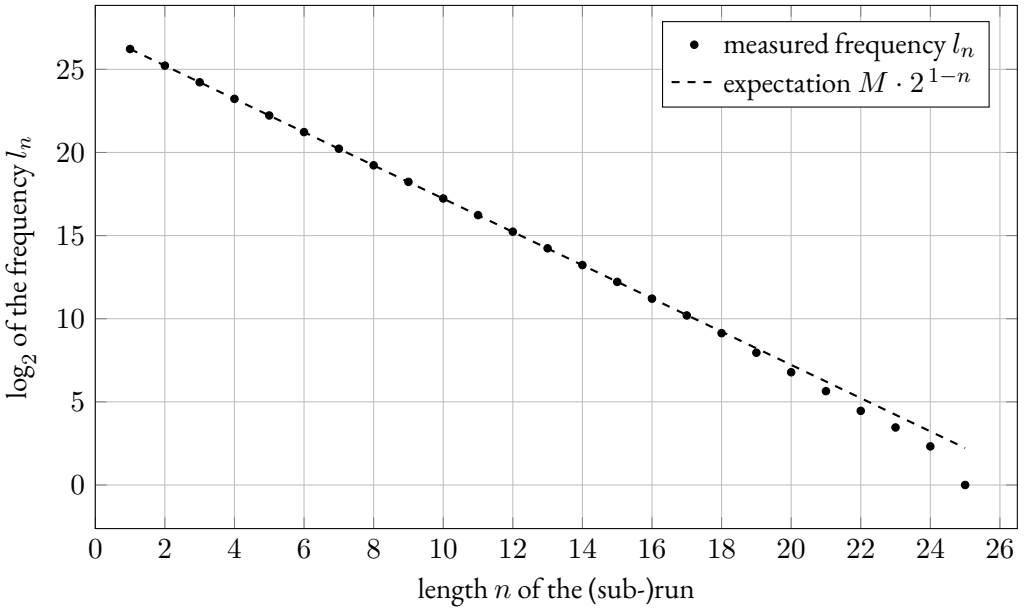


Figure 1: Logarithmic representation of the results of Table 1: $\log_2 l_n$ versus n (points) with expectation $M \cdot 2^{1-n}$ (dashed).

Equivalent to this is

$$ {}_N l_N = 1 \tag{16}$$

$$ {}_N l_{n-1} = 2\, {}_N l_n, \quad 2 \le n \le N \tag{17}$$

We point out that the cumulated distribution of a bit sequence does not suffice to characterise the sequence completely. Rather, this distribution corresponds to a class of bit sequences. Hence the following definition.

Definition 9 (Classes B_N)**.** We denote by B_N the set of sequences with longest run N and with exponentially decreasing cumulated distribution according to (15) or, equivalently, (16) and (17).

Definition 10 (Set of classes)**.** Set of the classes of bit sequences with exponentially decreasing cumulated distributions:

$$\mathcal{B}_{\mathcal{Z}} := \{B_N,\, 0 < N < \infty\} \tag{18}$$

6.2. The base distribution

Having fixed the basic definitions, we can turn to the study of the elements of an arbitrary B_N. Starting from the definition of the exponentially decreasing cumulated distribution, one may investigate whether the distribution of the frequencies of its runs *without sub-runs* follows a systematics. We thus subtract, from each frequency of the cumulated distribution ${}_N L$, the already offset sub-runs; the result we call the *base distribution*.

Definition 11 (Base distribution)**.** Let $(b_m) \in \mathcal{B}$ be a bit sequence with run-length sequence $(r_n)_{1 \le n \le R}$ and cumulated distribution $L = (l_p)_{1 \le p \le P}$. The *base distribution* $G = (g_p)_{1 \le p \le P}$ counts the runs by length,

$$g_p := \left|\{n \in \{1, \ldots, R\} :\, r_n = p\}\right|, \qquad 1 \le p \le P. \tag{19}$$

Cumulated and base distribution are linked by the triangular relation

$$l_p = \sum_{k=p}^{P} (k - p + 1) \cdot g_k, \qquad 1 \leq p \leq P, \tag{20}$$

for a run of length k contains exactly $k - p + 1$ sub-runs of length p. Because of the triangular structure of (20), G is uniquely determined by L; the recursive resolution from above,

$$g_P = l_P, \qquad g_p = l_p - \sum_{k=p+1}^{P} (k - p + 1) \cdot g_k \quad (p = P - 1, \ldots, 1), \tag{21}$$

is precisely the offsetting procedure described in the following.

For illustration we trace the resolution (21) on an example from B_5. In this case the values of ${}_5L = ({}_5l_i)_{0<i\leq 5}$ are

$${}_5l_i = 2^{5-i} \tag{22}$$

that is, explicitly:

- ${}_5l_1 = 16$,
- ${}_5l_2 = 8$,
- ${}_5l_3 = 4$,
- ${}_5l_4 = 2$,
- ${}_5l_5 = 1$.

The longest run has length 5 and is not a sub-run. We therefore subtract the sub-runs offset by the run of length 5 and obtain:

- 5 sub-runs 1,
- 4 sub-runs 2,
- 3 sub-runs 3,
- 2 sub-runs 4,
- 1 run 5.

Now the run 5 is offset, and with it the two runs 4 as well. The runs still to be accounted for are:

- ${}_5l_1 - 5 = 11$ runs 1,
- ${}_5l_2 - 4 = 4$ runs 2,
- ${}_5l_3 - 3 = 1$ run 3,
- ${}_5l_4 - 2 = 0$ runs 4,
- ${}_5l_5 - 1 = 0$ runs 5.

We now claim the run 3, since it is no longer present as a sub-run. This means

- 3 sub-runs 1,
- 2 sub-runs 2,
- 1 run 3.

Now the run 3 is offset. The runs still to be accounted for are:

- ${}_5l_1 - 5 - 3 = 8$ runs 1,
- ${}_5l_2 - 4 - 2 = 2$ runs 2,
- ${}_5l_3 - 3 - 1 = 0$ runs 3.

We now claim the runs 2. This means

- 4 sub-runs 1,
- 2 sub-runs 2.

Now the runs 2 are offset. The runs still to be accounted for are:

- ${}_5l_1 - 5 - 3 - 4 = 4$ runs 1.

Since runs of length 1 contain no sub-runs, they are offset immediately. The result of the resolution (21) is the base distribution

$$ {}_5G = (\ 4; 2; 1; 0; 1\ ). \tag{23}$$

One possible realisation of this distribution is

$$(\ 0101001100011111\ ) \text{ with } (\ \underbrace{0\,1\,0\,1}_{4}\ \underbrace{00\,11}_{2}\ \underbrace{000}_{1}\ \underbrace{\ }_{0}\ \underbrace{11111}_{1}\ ) \in B_5 \tag{24}$$

Of course, every other permutation of this sequence of runs is valid; the bit values are complemented where necessary. By way of example we repeat this computation for bit sequences of the classes B_1 through B_8:

$$\begin{aligned}
(\ 1\ )_{[B_1]} \\
(\ 0\ ;\ 1\ )_{[B_2]} \\
(\ 1\ ;\ 0\ ;\ 1\ )_{[B_3]} \\
(\ 2\ ;\ 1\ ;\ 0\ ;\ 1\ )_{[B_4]} \\
(\ 4\ ;\ 2\ ;\ 1\ ;\ 0\ ;\ 1\ )_{[B_5]} \\
(\ 8\ ;\ 4\ ;\ 2\ ;\ 1\ ;\ 0\ ;\ 1\ )_{[B_6]} \\
(\ 16\ ;\ 8\ ;\ 4\ ;\ 2\ ;\ 1\ ;\ 0\ ;\ 1\ )_{[B_7]} \\
(\ 32\ ;\ 16\ ;\ 8\ ;\ 4\ ;\ 2\ ;\ 1\ ;\ 0\ ;\ 1\ )_{[B_8]}
\end{aligned} \tag{25, 26}$$

We immediately note that the base distributions of the elements of the classes B_N exhibit a simple structure as N grows: the frequencies of the runs 2 through N shift one position to the right, and the frequency of the runs 1 doubles. We now underpin this experimental observation formally.

Theorem 12. *Let $N \geq 2$ and $({}_N b_m) \in B_N$. Then the base distribution $({}_N g_n)_{1 \leq n \leq N}$ of the runs of $({}_N b_m)$ satisfies*

$$ {}_N g_n = 2^{N-2-n}, \quad 1 \leq n \leq N-2, \tag{27}$$

$$ {}_N g_{N-1} = 0, \tag{28}$$

$$ {}_N g_N = 1. \tag{29}$$

In particular, ${}_N G$ satisfies the recursion ${}_N g_n = 2\, {}_N g_{n+1}$ for $1 \leq n \leq N-3$. (For $N = 1$ one has ${}_1 g_1 = 1$.)

Proof. Since the longest run has length N, we have $P = N$ and ${}_N l_n = 2^{N-n}$ for all $1 \leq n \leq N$ (equation (15)). We resolve the recursion (21) from top to bottom.

For $n = N$ one has ${}_N g_N =_N l_N = 2^0 = 1$, which is (29). For $n = N-1$ one has

$$ {}_N g_{N-1} =_N l_{N-1} - 2 \cdot_N g_N = 2 - 2 = 0, $$

which is (28).

For the remaining indices we prove the closed form (27) by downward induction over $n = N-2, \ldots, 1$. Subtracting equations (20) for n and $n+1$ yields

$$ {}_N l_n - {}_N l_{n+1} = {}_N g_n + \sum_{k=n+1}^{N} {}_N g_k, \qquad 1 \leq n \leq N-1, \tag{30}$$

for the contribution of a run of length $k > n$ to l_n exceeds its contribution to l_{n+1} by exactly 1.

Base case $n = N-2$: from (21) follows

$$ {}_N g_{N-2} =_N l_{N-2} - 2 \cdot \underbrace{{}_N g_{N-1}}_{=0} - 3 \cdot \underbrace{{}_N g_N}_{=1} = 4 - 3 = 1 = 2^{N-2-(N-2)}. $$

Induction step: let $n \leq N-3$ and suppose ${}_N g_k = 2^{N-2-k}$ has been shown for all $n < k \leq N-2$. From (30) and ${}_N l_n - {}_N l_{n+1} = 2^{N-n-1}$ follows, with the geometric sum,

$$ {}_N g_n = 2^{N-n-1} - \sum_{k=n+1}^{N-2} 2^{N-2-k} - \underbrace{{}_N g_{N-1}}_{=0} - \underbrace{{}_N g_N}_{=1} = 2^{N-n-1} - \left(2^{N-2-n} - 1\right) - 1 = 2^{N-2-n}. $$

This shows (27) for all $1 \leq n \leq N-2$ and hence the claim. □

Remark 13 (A peculiarity of the method). No element of B_N, $N \geq 2$, contains a run of length $N-1$: ${}_N g_{N-1} = 0$. The base distribution jumps directly from ${}_N g_{N-2} = 1$ to ${}_N g_N = 1$. This is not an irregularity but a particular, systematic property of the procedure: in the cumulation, the frequency share of length $N-1$ has been absorbed entirely by the sub-runs of the unique run of length N – ${}_N l_{N-1} = 2$ counts precisely its two sub-runs of length $N-1$ – and in the resolution from above this share is subtracted before an own run of length $N-1$ could become free.

Lemma 14 (Sum identity). *For all $m \geq 0$,*

$$ T_m := \sum_{k=1}^{m} k \cdot 2^{m-k} = 2^{m+1} - m - 2. \tag{31}$$

Proof. Induction over m. For $m = 0$ the sum is empty and $T_0 = 0 = 2^1 - 0 - 2$. In the step,

$$T_{m+1} = \sum_{k=1}^{m+1} k{\cdot}2^{m+1-k} = 2\sum_{k=1}^{m} k{\cdot}2^{m-k} + (m+1) = 2T_m + (m+1) = 2^{m+2} - (m+1) - 2. \quad \square$$

Corollary 15 (Converse and existence)**.** *If a bit sequence has the run distribution*

$$_N g_n = 2^{N-2-n}\ (1 \le n \le N-2), \qquad _N g_{N-1} = 0, \qquad _N g_N = 1, \tag{32}$$

then it lies in B_N. In particular, B_N is non-empty for every $N \ge 1$.

Proof. We have to show that the cumulated distribution is $_N l_n = 2^{N-n}$. From (20) and (32) follows for $1 \le n \le N-2$, with $m := N-2-n$,

$$\begin{aligned} {}_N l_n &= \sum_{k=n}^{N-2} (k-n+1)\cdot 2^{N-2-k} + (N-n+1)\cdot 1 \\ &= \sum_{j=0}^{m} (j+1)\cdot 2^{m-j} + (m+3) = \sum_{k=1}^{m+1} k\cdot 2^{m+1-k} + (m+3) \\ &= \left(2^{m+2} - m - 3\right) + (m+3) = 2^{m+2} = 2^{N-n}, \end{aligned}$$

where the penultimate step uses the sum identity (Lemma 14, $m \mapsto m+1$). For $n = N-1$ one has $_N l_{N-1} = 2 \cdot_N g_N = 2 = 2^{N-(N-1)}$ and for $n = N$ one has $_N l_N =_N g_N = 1$. The longest run has length N, hence the bit sequence lies in B_N. $\square$

6.3. The offsetting of sub-runs

The offsetting of sub-runs, as presented in the procedure for deriving the base distribution, can also be computed directly: the frequency $_N t_n$ of the offset sub-runs of length n is the sum of the sub-run contributions of all longer runs; a run of length i contains exactly $i - n + 1$ sub-runs of length n:

$$_N t_n = \sum_{i=n+1}^{N} (i-n+1) \cdot_N g_i, \qquad 1 \le n \le N. \tag{33}$$

With $_N t_n =_N l_n -_N g_n$ one obtains the general expression

$$_N l_n -_N g_n = \sum_{i=n+1}^{N} (i-n+1) \cdot_N g_i, \tag{34}$$

which holds for arbitrary base distributions and equivalently reformulates the triangular relation (20). For the exponentially decreasing cumulated distributions $_N l_n = 2^{N-n}$ and the base distribution of Theorem 12 we verify three cases.

♦ $n = N$: the sum in (33) is empty, hence

$$_N t_N = 0. \tag{35}$$

♦ $n = N-1$:

$$_N t_{N-1} = (N-(N-1)+1) \cdot \underbrace{{}_N g_N}_{=1} = 2 = 2^{N-(N-1)} - \underbrace{{}_N g_{N-1}}_{=0}. \tag{36}$$

♦ $1 \le n \le N-2$: with ${}_Nl_n = 2^{N-n}$ and ${}_Ng_n = 2^{N-2-n}$ one has

$$ {}_Nt_n = {}_N\, l_n - {}_N\, g_n = 2^{N-n} - 2^{N-2-n} = 3 \cdot 2^{N-2-n}. \tag{37}$$

From (37) follows, by an index shift, the shift structure also visible in the examples (25) through (26): for $1 \le n \le N-2$,

$$ {}_Nt_n = 3 \cdot 2^{N-2-n} = {}_{N-n+1}t_1, \tag{38}$$

for by (37) one has ${}_Mt_1 = 3 \cdot 2^{M-3}$, so for $M = N-n+1$ precisely the right-hand side of (38). The offset sub-runs of length n of a class B_N thus coincide with the offset sub-runs of length 1 of the class B_{N-n+1}.

In summary we record:

Lemma 16. *Let $N < \infty$. Then*

$$ 2^N = (N+1) + \left(1_{\{N>1\}} \cdot \sum_{n=0}^{N-2} (N-1-n) \cdot 2^n \right) \tag{39}$$

Proof. The substitution $k = N-1-n$ transforms the sum into

$$ \sum_{n=0}^{N-2} (N-1-n) \cdot 2^n = \sum_{k=1}^{N-1} k \cdot 2^{N-1-k} = T_{N-1}, $$

and by Lemma 14 ($m = N-1$) one has $T_{N-1} = 2^N - N - 1$, hence

$$ (N+1) + T_{N-1} = (N+1) + \left(2^N - N - 1\right) = 2^N. $$ □

Remark 17 (Total length). The right-hand side of (39) decomposes the bit length of a representative of B_{N+1} into the contributions of the individual runs: the unique run of length $N+1$ contributes $N+1$ bits, a run of length $k \le N-1$ occurs 2^{N-1-k} times and contributes k bits each; length N contributes nothing, for ${}_{N+1}g_N = 0$ (Remark 13). The lemma thus states: every representative of B_{N+1} has exactly 2^N bits, and every representative of B_N accordingly exactly 2^{N-1} bits.

7. Algorithms for Generating an Element of B_N

Motivated by the results of the preceding section we give an iterative algorithm computing a class representative of B_N for $N < \infty$.

```
Input: N, the length of the longest run
create an empty bit vector B
append a run of length N
for all i from N-2 down to 1
    append 2^(N-i-2) runs of length i to B
output B
```

Algorithm 3: Iterative generation of a sequence from B_N for an arbitrary N

The correctness of the algorithm follows immediately from Corollary 15: it generates precisely the run distribution (32), and every bit sequence with this run distribution lies in B_N.

For the recursive algorithm we use the following construction, which describes the passage from B_{N-1} to B_N.

Lemma 18 (Construction)**.** *Let $N \geq 3$ and $({}_{N-1}b_m) \in B_{N-1}$. Extend every run of $({}_{N-1}b_m)$ by exactly one bit and insert 2^{N-3} runs of length 1 at positions where the alternating construction is preserved (for instance one between each pair of adjacent runs and one at an end). Then the result lies in B_N.*

Proof. By Theorem 12, applied to B_{N-1}, the sequence $({}_{N-1}b_m)$ has

$$\sum_{n=1}^{N-3} 2^{N-3-n} + 1 = \left(2^{N-3} - 1\right) + 1 = 2^{N-3}$$

runs. After the extension the sequence has run lengths $k+1$ instead of k, as well as 2^{N-3} new runs of length 1. A run of length k contains $k-n+1$ sub-runs of length n; after the extension therefore $(k+1)-n+1 = k-(n-1)+1$. For the cumulated distribution L' of the extended sequence one thus has, for all $2 \leq n \leq N$,

$$l'_n = \sum_{k \geq n-1} (k-(n-1)+1) \cdot {}_{N-1}g_k = {}_{N-1}l_{n-1} = 2^{(N-1)-(n-1)} = 2^{N-n},$$

so the extension shifts the cumulated distribution by one position. The new runs of length 1 change only l'_1: by Remark 17, $({}_{N-1}b_m)$ has 2^{N-2} bits, the extension adds 2^{N-3} bits, and the new runs add another 2^{N-3} bits, hence

$$l'_1 = 2^{N-2} + 2^{N-3} + 2^{N-3} = 2^{N-1}.$$

The longest run has length N (extension of the run of length $N-1$), and $l'_n = 2^{N-n}$ holds for all $1 \leq n \leq N$; the result therefore lies in B_N. □

On the example $(0001) \in B_3$: extending all runs yields $(0000\,11)$ with runs of lengths 4 and 2; adding $2^{4-3} = 2$ runs of length 1, for instance by appending them, gives $(0000\,11\,0\,1)$, whose base distribution is $(2;1;0;1)$ of B_4.
Lemma 18 establishes the correctness of the following recursive algorithm for computing a class representative of B_N for $N < \infty$.

```
Input: an element of B_N with N > 2
repeat for every run
    determine the length of the run and extend it by 1 bit
    if the run had length 1, add 2 runs of length 1
output the result
```

Algorithm 4: Recursive computation of a bit sequence from a B_N

The cases $N = 1$ and $N = 2$ can be treated separately in an implementation; a tested reference implementation of all algorithms is given in Appendix A.

8. Composite Elements from Different B_N

Let $B \in \mathbb{N}$ be arbitrary with its unique binary representation

$$B = \sum_{n=1}^{N} b_n \cdot 2^{n-1}, \quad b_n \in \{0,1\},\ b_N = 1, \tag{40}$$

where b_1 is the least significant bit. By Remark 17 every representative of B_n has exactly 2^{n-1} bits. If we thus assign the class B_n to the bit b_n, the number B corresponds exactly to the composition of the classes $\{B_n : b_n = 1\}$, and the composite bit sequence has the length prescribed by B:

$$B = \sum_{n=1}^{N} b_n \cdot 2^{n-1} = \sum_{\substack{1 \le n \le N \\ b_n = 1}} |\text{representative of } B_n| \,. \tag{41}$$

We can thus identify, in a unique way, an arbitrary natural number B with a composition of pairwise distinct classes B_N of $\mathcal{B}_{\mathcal{Z}}$ determined uniquely by this number, and we obtain the composite frequency distribution ${}_BH_N$ of the runs of a bit sequence of the prescribed length B. We agree on the following definition.

Definition 19 (Random bit sequence)**.** Every bit sequence (b_n) whose frequency distribution of runs is an arbitrary composition of frequency distributions of pairwise distinct classes $B_N \in \mathcal{B}_{\mathcal{Z}}$ we call random.

Remark 20 (Classification)*.* This definition is to be read as a proposal: it does not replace any limit or axiomatic statements but fixes a purely combinatorial condition for finite sequences. Its relation to the probabilistic and the algorithmic concepts of randomness is discussed in Section 9.

The frequency of run length n of an arbitrary composition of pairwise distinct classes of $\mathcal{B}_{\mathcal{Z}}$ follows from Theorem 12: the class B_n contributes (for $b_n = 1$) a single run of length n, the class B_{n+1} none (for ${}_{n+1}g_n = 0$), and every class B_i with $i \ge n+2$ contributes $2^{\,i-n-2}$ runs of length n:

$${}_BH_n = b_n + \sum_{i=n+2}^{N} b_i \cdot 2^{\,i-n-2}, \qquad 1 \le n \le N. \tag{42}$$

The length of the bit sequence obtained in this way is therefore

$$\begin{aligned} \sum_{n=1}^{N} n \cdot_B H_n &= \sum_{n=1}^{N} n \cdot b_n + \sum_{n=1}^{N} \sum_{i=n+2}^{N} n \cdot b_i \cdot 2^{\,i-n-2} \\ &= \sum_{i=1}^{N} b_i \left(i + \sum_{n=1}^{i-2} n \cdot 2^{\,i-n-2} \right) = \sum_{i=1}^{N} b_i \,(i + T_{i-2}) \\ &= \sum_{i=1}^{N} b_i \cdot 2^{\,i-1} = B, \end{aligned} \tag{43}$$

using the sum identity (Lemma 14, $m = i-2$) in the penultimate step, for $T_{i-2} = 2^{i-1} - i$. The composition is thus consistent with the binary representation (40). We illustrate this state of affairs with

$$B = 101010101_2 = 341_{10} \tag{44}$$

in Table 2

We are now able to assign to every natural number a finite, uniquely determined subset $\{B_n\}_{n\in\mathbb{N}} \subset \mathcal{B}_{\mathcal{Z}}$ with $n_1 \ne n_2 \; \forall n_1, n_2$, and thereby to derive a uniquely determined composite frequency distribution of runs.

We now study the logarithmic values of the composite frequencies. Let

$$C_n := \sum_{i=n}^{N} {}_BH_i \tag{45}$$

n	b_n	1	2	3	4	5	6	7	8	9
1	1	1								
2	0	~~0~~	~~1~~							
3	1	1	0	1						
4	0	~~2~~	~~1~~	~~0~~	~~1~~					
5	1	4	2	1	0	1				
6	0	~~8~~	~~4~~	~~2~~	~~1~~	~~0~~	~~1~~			
7	1	16	8	4	2	1	0	1		
8	0	~~32~~	~~16~~	~~8~~	~~4~~	~~2~~	~~1~~	~~0~~	~~1~~	
9	1	64	32	16	8	4	2	1	0	1
	Σ	86	42	22	10	6	2	2	0	1
	n	1	2	3	4	5	6	7	8	9
	Π	86	84	66	40	30	12	14	0	9
Σ	341									

Table 2: Composite distribution for $B = 341$: column-wise contributions of the classes B_i with $b_i = 1$ to the frequency H_n of the runs of length n; row Σ: H_n, row Π: $n \cdot H_n$. Struck-out entries belong to classes with $b_i = 0$.

be the cumulated frequency of the runs of length at least n of the composite sequence, where we set $b_j := 0$ for $j > N$. From (42) follows, by interchanging the sums, the identity

$$C_n = b_n + b_{n+1} + \sum_{j=n+2}^{N} b_j \cdot 2^{\,j-n-1} = \frac{B_n + b_n}{2}, \tag{46}$$

where $B_n := \sum_{j=n}^{N} b_j \cdot 2^{\,j-n}$ is the natural number with binary representation $(b_N \ \ldots \ b_n)$. Since $b_N = 1$, one has $2^{N-n} \le B_n \le 2^{N-n+1} - 1$, hence the band statement

$$2^{N-n-1} \ \le \ C_n \ \le \ 2^{N-n}, \qquad \text{i.e.} \qquad N - n - 1 \ \le \ \log_2 C_n \ \le \ N - n. \tag{47}$$

Both bounds are sharp: if only b_N is set, then $C_n = 2^{N-n-1}$ and $\log_2 C_n = N - n - 1$ exactly; if all bits are set, then $C_n = 2^{N-n}$. Conversely, if a single bit b_m is changed from 0 to 1, then C_n grows, for $n \le m - 2$, by $1 + 2^{m-n-1}$ and, for $n \in \{m-1, m\}$, by 1; this increase is at most C_n itself (for $n \le m - 2$ because $1 + 2^{m-n-1} \le 2^{N-n-1} \le C_n$ with $m \le N - 1$ and $n \le N - 3$, for $n \in \{m-1, m\}$ because $C_n \ge 2^{N-n-1} \ge 1$). The logarithm to base two therefore grows by at most 1. This is the formal background of the experimental observation of Section 5: there the logarithmised frequencies (third column of Table 1) lie just below the theoretical values (fourth column).

9. Discussion

Only the frequency distribution of the runs of a bit sequence decides its membership in a class B_N with $N < \infty$. The consequence is that not only the seemingly irregular realisation (24) lies in B_5, but also its most regular variant 11111 0 111 0 11 0 11 0: not the arrangement of the runs but solely their frequency distribution is decisive. As a further illustration we may view every bit sequence as a binary number and compare it with any other bit sequence by the usual order relations. We take the

elements of B_5. The largest element is[2] (In the binary representation the most significant bit stands on the left.)

$$11111\,0\,111\,0\,11\,0\,11\,0 \tag{48}$$

One recognises immediately that the runs of length greater than one are sorted in decreasing order and are separated by runs of length one. The smallest element of B_5, on the other hand, is

$$00000\,1\,000\,1\,00\,1\,00\,1 \tag{49}$$

One recognises immediately that it is the binary complement of the largest element of B_5. Furthermore,

$$1\,00000\,1\,000\,1\,00\,1\,00 \tag{50}$$

is the smallest element of B_5 beginning with a one, and its binary complement

$$0\,11111\,0\,111\,0\,11\,0\,11 \tag{51}$$

is the largest element beginning with a zero. Moreover, B_5 contains no element between (50) and (51).

A certain structure in the classes B_N can thus be intuitively surmised. For membership in B_N this recognisability is irrelevant; only the frequency distribution of the runs decides.

Remark 21 (Cardinality). The classes B_N are large. An element of B_N ($N \geq 2$) is determined by the order of its $R = 2^{N-2}$ runs and the bit of the first run (Lemma 3); hence, with the base distribution of Theorem 12,

$$|B_N| = 2 \cdot \frac{R!}{\prod_{n=1}^{N} g_n!}, \qquad R = 2^{N-2}, \quad g_n = 2^{N-2-n}. \tag{52}$$

(The formula is confirmed for $N \leq 5$ by complete enumeration.) By Stirling's formula, (52) implies

$$\log_2 |B_N| = 2^{N-1} - \Theta\big(N^2\big)\,. \tag{53}$$

The description length of an element of B_N thus grows essentially with the full bit length 2^{N-1} of the sequence. Specifying an element requires practically the specification of the entire sequence, whereas verifying membership with the one-step test takes time linear in the sequence length. The fraction of the class among all sequences of length 2^{N-1} is $2^{-\Theta(N^2)}$ and tends to 0 – a complexity statement going beyond this trivial asymmetry of specification and verification, for instance a relation to $P \neq NP$ in the sense of cryptographic assumptions, cannot be derived from the combinatorics of the B_N, however.

Whether the classes $\mathcal{B}_{\mathcal{Z}}$ can thereby replace the probabilistic concept of randomness is a philosophical, not a combinatorial, question. The definition proposed here (Definition 19) is deliberately purely combinatorial and tailored to finite sequences; it stands alongside the probabilistic concept of randomness [4] and the algorithmic concept of randomness of Martin-Löf [7], which defines randomness of infinite sequences by passing all effective tests. Against this background the thesis may be permitted that it is not the randomness of probability theory that exists, but the arbitrariness of choosing an element of $\mathcal{B}_{\mathcal{Z}}$ – and Remark 21 shows that this arbitrariness is measurable: it grows with the full length of the chosen sequence.

[2] The spaces in the representation serve readability only and have no bearing on the content.

10. Summary and Outlook

We have introduced, with the one-step test, an evaluation procedure for finite bit sequences which cumulates the frequencies of runs and sub-runs. The experimental application to the output of `/dev/random` (78.3 million bits, a collection period of about fifteen months) shows an exponentially decreasing course, as is to be expected from the geometric distribution of the run lengths of independent bits; a chi-square goodness-of-fit test confirms this. The classes B_N of bit sequences with exactly exponentially decreasing cumulated distribution possess a strictly recursive base distribution (Theorem 12); from this result an iterative and a recursive generator, and, via the binary representation of natural numbers, a unique composition of the classes (Section 8).

A tested reference implementation of all four algorithms including the membership test for the classes B_N is given in Appendix A; the equivalence of the two variants of the one-step test is proved in Lemma 6, and the relation to the classical concepts of randomness and the cardinality statements are discussed in Section 9.

Open and subject of further work are:

- the extension of the experiment to further sources (`/dev/urandom`, pseudo- and hardware generators) and larger samples;
- improvements to and maintenance of the present text.

A. Reference Implementation

The Python module in Algorithm 5 implements the one-step test in both variants (Algorithms 1 and 2), the two generators (Algorithms 3 and 4) and the membership test for the classes B_N. The self-test verifies, for $N = 1, \ldots, 16$, that both generators produce only sequences whose cumulated distribution falls off exactly exponentially, that the recursive passage leads from B_N to B_{N+1} (Lemma 18), and that both variants of the one-step test yield identical results on pseudo-random bits (Lemma 6).

```
# Reference implementation of the algorithms of the paper
# "The Evaluation of Finite Bit Sequences".
#
#  - one_step(bits):      one-step test, variant for bit sequences
#                         (Algorithm 2)
#  - one_step_runs(r):    one-step test, variant for sequences of
   runs
#                         (Algorithm 1)
#  - b_n_iterative(N):    iterative generator for B_N (Algorithm 3)
#  - b_n_recursive(r):    recursive passage B_N -> B_{N+1} (Algorithm
    4)
#  - in_class(r, N):      membership test for B_N
#
# All functions operate on sequences of bits (0/1) or on sequences
# of run lengths. The return values are the cumulated distribution
# or the sequence of run lengths, respectively.


def one_step(bits):
    """Cumulated distribution of the runs and sub-runs of a bit
   sequence.

    For every bit of the sequence, all positions 1..z are counted,
    where z is the length of the run begun at that bit.
```

```
    """
    L, z, previous = [], 0, None
    for b in bits:
        if b == previous:
            z += 1
        else:
            z = 1
            previous = b
        for i in range(z):
            if i == len(L):
                L.append(0)
            L[i] += 1
    return L


def one_step_runs(runs):
    """Cumulated distribution from the sequence of run lengths.

    Every run of length l increases position i (1 <= i <= l)
    by l - i + 1.
    """
    L = []
    for l in runs:
        while len(L) < l:
            L.append(0)
        for i in range(1, l + 1):
            L[i - 1] += l - i + 1
    return L


def b_n_iterative(N):
    """Class representative of B_N as a sequence of run lengths
    (Algorithm 3): one run of length N, then for i = N-2, ..., 1
    exactly 2^(N-i-2) runs of length i each.
    """
    runs = [N]
    for i in range(N - 2, 0, -1):
        runs += [i] * (2 ** (N - i - 2))
    return runs


def b_n_recursive(runs):
    """Passage B_N -> B_{N+1} (Algorithm 4).

    Extends every run by one bit and appends 2^(N-2) runs of length
    1
    (N = longest run of the input). The cases N = 1 and N = 2
    require
    no augmentation.
    """
    N = max(runs)
    extra = 0 if N < 2 else 2 ** (N - 2)
    return [l + 1 for l in runs] + [1] * extra
```

```
def in_class(runs, N):
    """Membership test: does the sequence of run lengths lie in B_N?

    Checks whether the cumulated distribution falls off exactly
    exponentially: l_n = 2^(N-n) for all 1 <= n <= N.
    """
    L = one_step_runs(runs)
    return len(L) == N and all(
        L[n - 1] == 2 ** (N - n) for n in range(1, N + 1)
    )


def runs_to_bits(runs, start=0):
    """Realisation of a sequence of run lengths as a bit sequence.""
    "
    bits, b = [], start
    for l in runs:
        bits += [b] * l
        b = 1 - b
    return bits


def runs_of(bits):
    """Run lengths of a bit sequence."""
    runs, z = [], 1
    for i in range(1, len(bits)):
        if bits[i] == bits[i - 1]:
            z += 1
        else:
            runs.append(z)
            z = 1
    runs.append(z)
    return runs


if __name__ == "__main__":
    # Self-test: the generators produce elements of B_N, and both
    # variants of the one-step test agree.
    import random

    for N in range(1, 17):
        r = b_n_iterative(N)
        assert in_class(r, N), N
        assert one_step(runs_to_bits(r)) == one_step_runs(r), N
        if N >= 2:
            assert in_class(b_n_recursive(r), N + 1), N

    random.seed(0)
    for _ in range(20):
        bits = [random.randint(0, 1) for _ in range(5000)]
        assert one_step(bits) == one_step_runs(runs_of(bits))

    print("All self-tests passed.")
```

Algorithm 5: einschritttest.py: reference implementation

References

[1] William Feller. *An Introduction to Probability Theory and Its Applications*. 3rd ed. Vol. 1. New York: Wiley, 1968.

[2] Zvi Gutterman and Dahlia Malkhi. "Hold the RESQ: Analysis of the Linux Random Number Generator". In: *Proceedings of the 11th ACM Conference on Computer and Communications Security (CCS)*. New York: ACM, 2004, pp. 264–275.

[3] Donald E. Knuth. *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*. 3rd ed. Reading, MA: Addison-Wesley, 1997.

[4] A. N. Kolmogoroff. *Grundbegriffe der Wahrscheinlichkeitsrechnung*. Berlin: Springer, 1933.

[5] Pierre L'Ecuyer. "Uniform Random Number Generation". In: *Annals of Operations Research* 53 (1994), pp. 77–120.

[6] George Marsaglia. *DIEHARD: A Battery of Tests of Randomness*. Tech. rep. Tallahassee, FL: Florida State University, 1996.

[7] Per Martin-Löf. "The Definition of Random Sequences". In: *Information and Control* 9.6 (1966), pp. 602–619.

[8] Karl R. Popper. *Logik der Forschung*. Wien: Julius Springer, 1935.

[9] Andrew Rukhin et al. *A Statistical Test Suite for Random and Randomized Cryptographic Algorithms*. NIST Special Publication 800-22 Rev. 1a. National Institute of Standards and Technology, 2010.

[10] A. Wald and J. Wolfowitz. "On a test whether two samples are from the same population". In: *The Annals of Mathematical Statistics* 11.2 (1940), pp. 147–162.